

Maven -Hands-on

Apache Maven = A build tool from Apache that acts like an expert in managing and building your project

Real-time workflow:

1. Install Java + Maven
2. Create project (**archetype**)
3. Modify code & **pom.xml**
4. Run:
 - **mvn clean**
 - **mvn compile**
 - **mvn test**
 - **mvn package**

```
mvn clean      → Deletes old build files
mvn compile    → Compiles source code
mvn test       → Runs test cases
mvn package    → Creates .jar/.war file
```

5. Run JAR

Full Structure Example

```
project/
├── pom.xml
└── src/
    ├── main/
    │   ├── java/
    │   └── resources/
    └── test/
        ├── java/
        └── resources/
```

Real-time analogy

```
compile → test → package → deploy
```

- `pom.xml` → Project manager
 - `main` → actual application
 - `test` → validation code
 - `src/main` → used in compile /Actual product (car engine, design)
 - `src/test` → used in test phase only /Quality testing team
-
- In Maven, `pom.xml` manages dependencies and build configuration, `src/main` contains production application code, and `src/test` contains test code to validate the application, ensuring clean separation and efficient build lifecycle management.

Hands-on :

1. System Setup

Update system

```
sudo yum update -y
```

- Updates all packages in the system

Install Java

```
yum install java-21-amazon-corretto -y
```

- Installs Java 21 (Amazon Corretto)

```
java -version
```

- Verify Java installation

Install Maven

```
yum install maven -y
```

- Installs Maven

```
mvn -v
```

- Check Maven version
-

2. Create Maven Project

```
mvn archetype:generate \  
-DgroupId=com.example \  
-DartifactId=demo-app \  
-DarchetypeArtifactId=maven-archetype-quickstart \  
-DinteractiveMode=false
```

- Generates a **ready-made project structure**
 - **groupId** → company/package name
 - **artifactId** → project name
 - **archetype** → template
-

3. Navigate Project

```
ll  
cd demo-app  
ls -R
```

- View project files and structure
-

4. Edit Code & Config

```
vi src/main/java/com/example/App.java
```

- Modify main application code

```
vi pom.xml
```

- Modify project configuration

Add Java version:

```
<properties>  
  <maven.compiler.source>21</maven.compiler.source>  
  <maven.compiler.target>21</maven.compiler.target>  
</properties>
```

5. Build Lifecycle Commands

Clean

```
mvn clean
```

- Deletes `target/` folder (old build files)

Compile

```
mvn compile
```

- Converts `.java` → `.class`

Clean + Compile

```
mvn clean compile
```

Test

```
mvn test
```

- Runs unit tests from `src/test`

Package

```
mvn package
```

- Creates `.jar` file in `target/`

Install

```
mvn install
```

- Installs jar into local Maven repo (`.m2`)

6. Fix Java Version Issues (Important)

```
sudo dnf install java-21-amazon-corretto -y  
sudo alternatives --config java
```

- Select correct Java version

Set JAVA_HOME

```
export JAVA_HOME=/usr/lib/jvm/java-21-amazon-corretto  
export PATH=$JAVA_HOME/bin:$PATH  
echo "export JAVA_HOME=/usr/lib/jvm/java-21-amazon-corretto" >>  
~/.bashrc  
echo "export PATH=$JAVA_HOME/bin:$PATH" >> ~/.bashrc
```

- Ensures Maven uses correct Java

7. Advanced Build Commands

```
mvn clean package -DskipTests
```

- Skip tests while building

```
mvn clean install -U
```

- Force update dependencies

8. Check Output

```
ls target
```

- Shows generated `.jar`

9. Run Application

```
java -cp target/demo-app-1.0-SNAPSHOT.jar com.example.App
```

- Runs Java program using classpath

10. Explore Project Structure

```
ls src
ls src/main/
ls src/test/
```

- Understand Maven folder structure

11. Navigate Code

```
cd src/main/java/com/example
ls
vi App.java
```

- Access and edit source code

12. Misc Commands

```
history
```

- Shows previously executed commands

One-Line Interview Answer :

- ★ Maven commands automate build lifecycle: clean removes old builds, compile builds code, test validates it, package creates a JAR, and install stores it in the local repository.